

```

    }
    }
    }
    int ans = 0;
    for (int i = 0; i < N; i++) if (vis[i]) ans++;
    cout << ans << "\n";
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.2.2 节例题 5
同类题目：Sweet Dream, UVa 10969
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表6-2所示。

表6-2 与圆有关的计算问题例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 6-3	UVa 12304	2D Geometry 110 in 1	圆和直线的各种问题	刘汝佳
例 6-4	UVa 1308	Viva Confetti	圆的综合问题	刘汝佳

6.3 二维几何常用算法

本节通过一些经典题目提供了凸包、半平面交、平面区域等二维几何常用的算法实现。

例6-5 【凸包】包装木板 (Board Wrapping, UVa 10652)

有 n ($2 \leq n \leq 600$) 块矩形木板，你的任务是用一个面积尽量小的凸多边形把它们包起来，并计算出木板占整个包装面积的百分比，如图6-4所示。

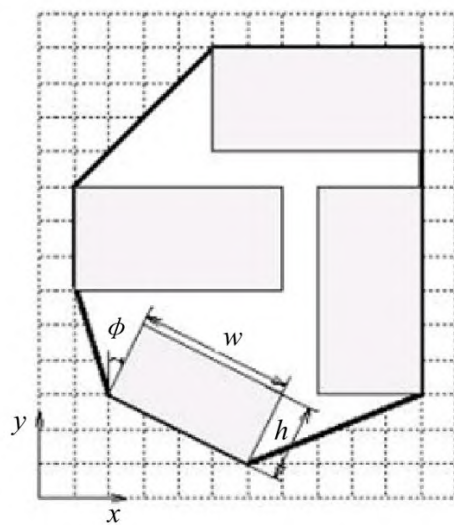


图6-4 包装木板

【代码实现】

```

// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const double PI = acos(-1.0);
double torad(double deg) { return deg / 180 * PI; }

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
};

istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
ostream& operator<<(ostream& os, const Point& p) { return os<<p.x<<" "<<p.y; }
typedef Point Vector;

Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }
Vector Rotate(const Vector& A, double rad)
{ return Vector(A.x*cos(rad) - A.y*sin(rad), A.x*sin(rad) + A.y*cos(rad)); }
bool operator < (const Point& p1, const Point& p2)
{ return p1.x < p2.x || (p1.x == p2.x && p1.y < p2.y); }
bool operator == (const Point& p1, const Point& p2)
{ return p1.x == p2.x && p1.y == p2.y; }

/* 点集凸包, 如果不希望在凸包的边上有输入点, 可把两个≤改成<。如果不介意点集被修改, 可以改成传递引用 */
vector<Point> ConvexHull(vector<Point> p) {
    sort(p.begin(), p.end()); // 预处理, 删除重复点
    p.erase(unique(p.begin(), p.end()), p.end());

    int n = p.size();
    int m = 0;
    vector<Point> ch(n + 1);
    for (int i = 0; i < n; i++) {
        while (m > 1 && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    int k = m;
    for (int i = n - 2; i >= 0; i--) {
        while (m > k && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    if (n > 1) m--;
    ch.resize(m);
}

```

```

    return ch;
}

// 多边形的有向面积
double PolygonArea(const vector<Point>& p) {
    double area = 0;
    int n = p.size();
    for (int i = 1; i < n - 1; i++)
        area += Cross(p[i] - p[0], p[i + 1] - p[0]);
    return area / 2;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T; cin >> T;
    for (int t = 0, n; t < T; t++) {
        double areal = 0;
        cin >> n;
        vector<Point> P;
        for (int i = 0; i < n; i++) {
            Point o;
            double w, h, j, ang;
            cin >> o >> w >> h >> j;
            ang = -torad(j);
            P.push_back(o + Rotate(Vector(-w / 2, -h / 2), ang));
            P.push_back(o + Rotate(Vector(w / 2, -h / 2), ang));
            P.push_back(o + Rotate(Vector(-w / 2, h / 2), ang));
            P.push_back(o + Rotate(Vector(w / 2, h / 2), ang));
            areal += w * h;
        }
        double area2 = PolygonArea(ConvexHull(P));
        printf("%.11f %%\n", areal * 100 / area2);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.2.2 节例题 6
同类题目：The Great Divide, UVa 10256
*/

```

例6-6 【凸包；直线的一般式】飞机场 (Airport, UVa 11168)

给出平面上的 n ($n \leq 10\,000$) 个点，找一条直线，使得所有点在直线的同侧（也可以在直线上），且到直线的距离之和尽量小。所有坐标均为绝对值不超过80 000的整数。输出距离之和的最小值。

【代码实现】

```
// 刘汝佳  
#include <bits/stdc++.h>
```

```

using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
};

istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
ostream& operator<<(ostream& os, const Point& p) { return os << p.x << " " <<
p.y; }
typedef Point Vector;
Vector operator - (const Point& A, const Point& B)
{return Vector(A.x - B.x, A.y - B.y);}
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }
bool operator < (const Point& p1, const Point& p2)
{ return p1.x < p2.x || (p1.x == p2.x && p1.y < p2.y); }
bool operator == (const Point& p1, const Point& p2)
{ return p1.x == p2.x && p1.y == p2.y; }
// 点集凸包, 如果不希望在凸包的边上有输入点, 把两个≤改成<
// 如果不介意点集被修改, 可以改成传递引用
vector<Point> ConvexHull(vector<Point> p) {
    sort(p.begin(), p.end()); // 预处理, 删除重复点
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = p.size(), m = 0;
    vector<Point> ch(n + 1);
    for (int i = 0; i < n; i++) {
        while (m > 1 && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    int k = m;
    for (int i = n - 2; i >= 0; i--) {
        while (m > k && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    if (n > 1) m--;
    ch.resize(m);
    return ch;
}

// 过两点p1, p2的直线, 一般方程为ax+by+c=0, (x2-x1)(y-y1) = (y2-y1)(x-x1)
void getLineGeneralEquation(const Point& p1, const Point& p2,
                           double& a, double& b, double& c) {
    a = p2.y - p1.y, b = p1.x - p2.x, c = -a * p1.x - b * p1.y;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T; cin >> T;
    for (int kase = 1, n; kase <= T; kase++) {

```

```

cin >> n;
vector<Point> P;
double sumx = 0, sumy = 0;
Point pt;
for (int i = 0; i < n; i++)
    cin >> pt, sumx += pt.x, sumy += pt.y, P.push_back(pt);
vector<Point> ch = ConvexHull(P);
int m = ch.size();
double ans = 1e9;
if (m <= 2) ans = 0; // 凸包退化成点或线段，则答案为0
else
    for (size_t i = 0; i < ch.size(); i++) {
        double a, b, c;
        getLineGeneralEquation(ch[i], ch[(i + 1) % ch.size()], a, b, c);
        ans = min(ans, fabs(a*sumx + b*sumy + c * n) / sqrt(a*a + b*b));
    }
printf("Case #%d: %.3lf\n", kase, ans / n);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.3.2 节例题 7
同类题目：A highway and the seven dwarfs, POJ 1912
*/

```

例6-7 【凸包；旋转卡壳】正方形（Square, Seoul 2009, UVa 1453）

给定平面上的 n （ $1 \leq n \leq 100\,000$ ）个边平行于坐标轴的正方形，在它们的顶点中找出两个欧几里得距离最大的点。如图6-5所示，距离最大的是 S_1 的左下角和 S_8 的右上角。正方形可以重合或者交叉。

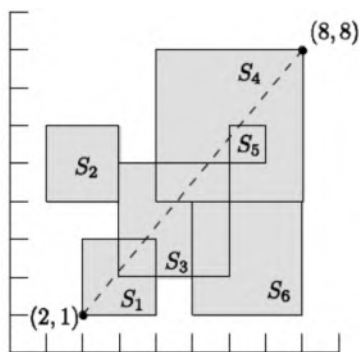


图6-5 正方形

你的任务是输出这个最大距离的平方。

【代码实现】

```
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

struct Point {
    int x, y;
    Point(int x = 0, int y = 0) : x(x), y(y) {}
};

typedef Point Vector;
istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
ostream& operator<<(ostream& os, const Point& p) {
    return os << p.x << " " << p.y;
}
Vector operator-(const Point& A, const Point& B)
```



```

{ return Vector(A.x - B.x, A.y - B.y); }
int Cross(const Vector& A, const Vector& B) { return A.x * B.y - A.y * B.x; }
int Dot(const Vector& A, const Vector& B) { return A.x * B.x + A.y * B.y; }
int Dist2(const Point& A, const Point& B) {
    return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) * (A.y - B.y);
}
bool operator<(const Point& p1, const Point& p2) {
    return p1.x < p2.x || (p1.x == p2.x && p1.y < p2.y);
}
bool operator==(const Point& p1, const Point& p2) {
    return p1.x == p2.x && p1.y == p2.y;
}

vector<Point> ConvexHull(vector<Point>& p) { // 点集凸包
    sort(p.begin(), p.end()); // 预处理, 删除重复点
    p.erase(unique(p.begin(), p.end()), p.end());

    int n = p.size();
    int m = 0;
    vector<Point> ch(n + 1);
    for (int i = 0; i < n; i++) {
        while (m > 1 && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    int k = m;
    for (int i = n - 2; i >= 0; i--) {
        while (m > k && Cross(ch[m - 1] - ch[m - 2], p[i] - ch[m - 2]) <= 0) m--;
        ch[m++] = p[i];
    }
    if (n > 1) m--;
    ch.resize(m);
    return ch;
}

int diameter2(vector<Point>& points) { // 返回点集直径的平方
    vector<Point> p = ConvexHull(points);
    int n = p.size();
    if (n == 1) return 0;
    if (n == 2) return Dist2(p[0], p[1]);
    p.push_back(p[0]); // 免得取模
    int ans = 0;
    for (int u = 0, v = 1; u < n; u++) {
        for (;;) { // 一条直线贴住边 p[u]-p[u+1]
            // 当 Area(p[u], p[u+1], p[v+1]) <= Area(p[u], p[u+1], p[v]) 时停止旋转
            // 即 Cross(p[u+1]-p[u], p[v+1]-p[u]) - Cross(p[u+1]-p[u], p[v]-p[u]) <= 0
            // 根据 Cross(A,B) - Cross(A,C) = Cross(A,B-C)
            // 化简得 Cross(p[u+1]-p[u], p[v+1]-p[v]) <= 0
            int diff = Cross(p[u + 1] - p[u], p[v + 1] - p[v]);
            if (diff <= 0) {

```

```

        ans = max(ans, Dist2(p[u], p[v])); // u 和 v 是对踵点
        if (diff == 0)
            ans = max(ans, Dist2(p[u], p[v + 1])); // diff==0 时 u 和 v+1 也是对踵点
        break;
    }
    v = (v + 1) % n;
}
}
return ans;
}

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    int T;
    cin >> T;
    for (int t = 0, n; t < T; t++) {
        cin >> n;
        vector<Point> ps;
        for (int i = 0, x, y, w; i < n; i++) {
            cin >> x >> y >> w;
            ps.push_back(Point(x, y)), ps.push_back(Point(x + w, y));
            ps.push_back(Point(x, y + w)), ps.push_back(Point(x + w, y + w));
        }
        printf("%d\n", diameter2(ps));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.3.2 节例题 9
同类题目：Making Perimeter of the Convex Hull Shortest, ACM/ICPC 筑波 2017, Aizu 1381
*/

```

例6-8 【二分法；半平面交】离海最远的点（Most Distant Point from the Sea, Tokyo 2007, UVa 1396）

在大海的中央，有一个凸 n ($3 \leq n \leq 100$) 边形的小岛。你的任务是求出岛上离海最远的点，输出它到海的距离。

【代码实现】

```
// 刘汝佳
#include <cmath>
#include <cstdio>
#include <vector>
#include <algorithm>
using namespace std;

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
};
```

```

typedef Point Vector;
Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator * (const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }
double Dot(const Vector& A, const Vector& B)
{ return A.x * B.x + A.y * B.y; }
double Cross(const Vector& A, const Vector& B)
{ return A.x * B.y - A.y * B.x; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
Vector Normal(const Vector& A)
{ double L = Length(A); return Vector(-A.y / L, A.x / L); }

double PolygonArea(vector<Point> p) {
    int n = p.size();
    double area = 0;
    for (int i = 1; i < n - 1; i++)
        area += Cross(p[i] - p[0], p[i + 1] - p[0]);
    return area / 2;
}

// 有向直线，它的左边就是对应的半平面
struct Line {
    Point P;          // 直线上任意一点
    Vector v;          // 方向向量
    double ang;        // 极角，即从 x 正半轴旋转到向量 v 所需要的角（弧度）
    Line() {}
    Line(const Point& P, const Vector& v): P(P), v(v) { ang = atan2(v.y, v.x); }
    bool operator < (const Line& L) const {
        return ang < L.ang;
    }
};

// 点 p 在有向直线 L 的左边（线上不算）
bool OnLeft(const Line& L, const Point& p) {
    return Cross(L.v, p - L.P) > 0;
}

// 两直线交点，假定交点唯一存在
Point GetLineIntersection(const Line& a, const Line& b) {
    Vector u = a.P - b.P;
    double t = Cross(b.v, u) / Cross(a.v, b.v);
    return a.P + a.v * t;
}

const double INF = 1e8, eps = 1e-6;

```

```

// 半平面交主过程
vector<Point> HalfplaneIntersection(vector<Line> L) {
    int n = L.size();
    sort(L.begin(), L.end()); // 按极角排序

    int first, last; // 双端队列的第一个元素和最后一个元素的下标
    vector<Point> p(n); // p[i]为q[i]和q[i+1]的交点
    vector<Line> q(n); // 双端队列
    vector<Point> ans; // 结果

    q[first = last = 0] = L[0]; // 双端队列初始化为只有一个半平面 L[0]
    for (int i = 1; i < n; i++) {
        while (first < last && !OnLeft(L[i], p[last - 1])) last--;
        while (first < last && !OnLeft(L[i], p[first])) first++;
        q[++last] = L[i];
        if (fabs(Cross(q[last].v, q[last-1].v)) < eps) { // 两向量平行且同向, 取内侧的一个
            last--;
            if (OnLeft(q[last], L[i].P)) q[last] = L[i];
        }
        if (first < last) p[last - 1] = GetLineIntersection(q[last - 1], q[last]);
    }
    while (first < last && !OnLeft(q[first], p[last - 1])) last--; // 删除无用平面
    if (last - first <= 1) return ans; // 空集
    p[last] = GetLineIntersection(q[last], q[first]); // 计算首尾两个半平面的交点

    // 从 deque 复制到输出中
    for (int i = first; i <= last; i++) ans.push_back(p[i]);
    return ans;
}

int main() {
    for (int n, x, y; scanf("%d", &n) == 1 && n;) {
        vector<Vector> p, v, normal;
        for (int i = 0; i < n; i++) scanf("%d%d", &x, &y), p.push_back(Point(x, y));
        if (PolygonArea(p) < 0) reverse(p.begin(), p.end());

        for (int i = 0; i < n; i++)
            v.push_back(p[(i + 1) % n] - p[i]), normal.push_back(Normal(v[i]));
        double left = 0, right = 20000;
        while (right - left > eps) {
            vector<Line> L;
            double mid = left + (right - left) / 2;
            for (int i = 0; i < n; i++) L.push_back(Line(p[i] + normal[i]*mid, v[i]));
            vector<Point> poly = HalfplaneIntersection(L);
            if (poly.empty()) right = mid; else left = mid;
        }
        printf("%.6lf\n", left);
    }
}

```

```

    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.3.3 节例题 10
同类题目：Triathlon, POJ 1755
*/

```

例6-9 【平面区域；图论模型】怪物陷阱 (Monster Trap, Aizu 2003, POJ 2048)

给出 n ($1 \leq n \leq 100$) 条线段障碍，你的任务是判断怪物能否逃到无穷远处。如图6-6所示，其中图6-6 (a) 中怪物无法逃出，图6-6 (b) 中怪物可以逃出。

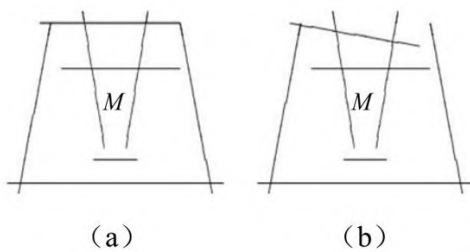


图6-6 怪物陷阱

对于每组数据，如果怪物成功被困住，输出yes，否则（即可以逃到无穷远处）输出no。

【代码实现】

```

// 刘汝佳
#include <iostream>
#include <vector>
#include <cmath>
#include <cstring>
#include <algorithm>
using namespace std;

const double eps = 1e-12;
double dcmp(double x) {
    if (fabs(x) < eps) return 0; else return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) { }
};

istream& operator>>(istream& is, Point& p) { return is >> p.x >> p.y; }
typedef Point Vector;
Vector operator + (const Point& A, const Point& B) { return Vector(A.x + B.x,
A.y + B.y); }
Vector operator - (const Point& A, const Point& B) { return Vector(A.x - B.x,
A.y - B.y); }
Vector operator * (const Point& A, double v) { return Vector(A.x * v, A.y * v); }
Vector operator / (const Point& A, double v) { return Vector(A.x / v, A.y / v); }

double Cross(const Vector& A, const Vector& B) { return A.x * B.y - A.y * B.x; }
double Dot(const Vector& A, const Vector& B) { return A.x * B.x + A.y * B.y; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
bool operator < (const Point& p1, const Point& p2)
{ return p1.x < p2.x || (p1.x == p2.x && p1.y < p2.y); }
bool operator == (const Point& p1, const Point& p2)

```

```

{ return p1.x == p2.x && p1.y == p2.y; }

bool SegmentProperIntersection(const Point& a1, const Point& a2,
                               const Point& b1, const Point& b2) {
    double c1 = Cross(a2 - a1, b1 - a1), c2 = Cross(a2 - a1, b2 - a1),
           c3 = Cross(b2 - b1, a1 - b1), c4 = Cross(b2 - b1, a2 - b1);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

bool OnSegment(const Point& p, const Point& a1, const Point& a2) {
    return dcmp(Cross(a1 - p, a2 - p)) == 0
        && dcmp(Dot(a1 - p, a2 - p)) < 0;
}

const int maxv = 200 + 5;
int V;
int G[maxv][maxv], vis[maxv];

bool dfs(int u) {
    if (u == 1) return true;           // 1是终点
    vis[u] = 1;
    for (int v = 0; v < V; v++)
        if (G[u][v] && !vis[v] && dfs(v)) return true;
    return false;
}

const int maxn = 100 + 5;
int n;
Point p1[maxn], p2[maxn];

// 在任何一条线段的中间（在端点不算）
bool OnAnySegment(Point p) {
    for (int i = 0; i < n; i++)
        if (OnSegment(p, p1[i], p2[i])) return true;
    return false;
}

// 与任何一条线段规范相交
bool IntersectWithAnySegment(Point a, Point b) {
    for (int i = 0; i < n; i++)
        if (SegmentProperIntersection(a, b, p1[i], p2[i])) return true;
    return false;
}

bool find_path() {
    // 构图
    vector<Point> vertices;
    vertices.push_back(Point(0, 0));           // 起点
    vertices.push_back(Point(1e5, 1e5));       // 终点

```



```

for (int i = 0; i < n; i++) {
    if (!OnAnySegment(p1[i])) vertices.push_back(p1[i]);
    if (!OnAnySegment(p2[i])) vertices.push_back(p2[i]);
}
V = vertices.size();
memset(G, 0, sizeof(G)), memset(vis, 0, sizeof(vis));
for (int i = 0; i < V; i++)
    for (int j = i + 1; j < V; j++)
        if (!IntersectWithAnySegment(vertices[i], vertices[j]))
            G[i][j] = G[j][i] = 1;
return dfs(0);
}

int main() {
    Point a, b;
    while (cin >> n && n) {
        for (int i = 0; i < n; i++) {
            cin >> a >> b;
            Vector v = b - a;
            v = v / Length(v);
            p1[i] = a - v * 1e-6, p2[i] = b + v * 1e-6;
        }
        if (find_path()) cout << "no\n"; else cout << "yes\n";
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 4.3.4 节例题 13
同类题目：Street Crossing EXTREME, UVa 11595
*/

```

例6-10 【平面区域；卷包裹法】找边界（Find the Border, NEERC 2004, UVa 1340）

一条可自交的封闭折线将平面分成了若干个区域，其中无限大的那个区域称为折线的外部区域，所有有界区域和折线本身合称内部区域（在图6-7中用阴影表示）。你的任务是求出内部区域的轮廓。

为了确保答案唯一，你求出的轮廓应当满足以下几个条件：不自交（但可以自

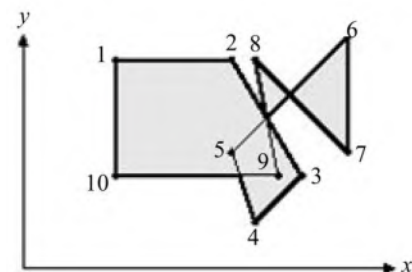


图6-7 求内部区域的轮廓

己和自己接触)；相邻两点不重合；相邻两条线段不共线；当按照轮廓线顺序访问它的各个顶点时，内部区域总是在边的左侧。

输入包含多组数据。每组数据：第一行为折线顶点数 n ($3 \leq n \leq 100$)；接下来是 n 对 $0 \sim 100$ 的整数，即折线上各顶点的坐标，所有顶点坐标均不同，且不会有顶点在另外两个顶点之间的线段上，相邻两条线段不共线。输入结束标志为文件结束符 (EOF)。

对于每组数据，首先输出一行，包含一个整数 m ，即轮廓线上的顶点个数，接下来是各顶点的坐标。

【代码实现】

```

// 刘汝佳
// 注意：本题可以直接使用“卷包裹”法求出外轮廓
// 本程序只是为了演示 PSLG 的实现
#include <vector>
#include <cassert>
#include <cmath>
#include <cstring>
#include <cstdio>
#include <algorithm>

using namespace std;

const double eps = 1e-8;
double dcmp(double x) {
    if (fabs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0): x(x), y(y) {}
};

typedef Point Vector;
Vector operator + (const Vector& A, const Vector& B)
{ return Vector(A.x + B.x, A.y + B.y); }
Vector operator - (const Point& A, const Point& B)
{ return Vector(A.x - B.x, A.y - B.y); }
Vector operator * (const Vector& A, double p)
{ return Vector(A.x * p, A.y * p); }

// 理论上这个“小于”运算符是错的，因为可能有 3 个点 a, b, c
// a 和 b 很接近（即 a<b 和 b<a 都不成立），b 和 c 很接近，但 a 和 c 不接近
// 所以使用这种“小于”运算符的前提是能排除上述情况
bool operator < (const Point& a, const Point& b)
{ return dcmp(a.x - b.x) < 0 || (dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) < 0); }
bool operator == (const Point& a, const Point& b)
{ return dcmp(a.x - b.x) == 0 && dcmp(a.y - b.y) == 0; }

double Dot(const Vector& A, const Vector& B) { return A.x * B.x + A.y * B.y; }
double Cross(const Vector& A, const Vector& B) { return A.x * B.y - A.y * B.x; }
double Length(const Vector& A) { return sqrt(Dot(A, A)); }
typedef vector<Point> Polygon;

Point GetLineIntersection(const Point& P, const Vector& v,
                          const Point& Q, const Vector& w) {

```

```

    Vector u = P - Q;
    double t = Cross(w, u) / Cross(v, w);
    return P + v * t;
}

bool SegmentProperIntersection(const Point& a1, const Point& a2,
                                const Point& b1, const Point& b2) {
    double c1 = Cross(a2 - a1, b1 - a1), c2 = Cross(a2 - a1, b2 - a1),
           c3 = Cross(b2 - b1, a1 - b1), c4 = Cross(b2 - b1, a2 - b1);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

bool OnSegment(const Point& p, const Point& a1, const Point& a2) {
    return dcmp(Cross(a1 - p, a2 - p)) == 0
        && dcmp(Dot(a1 - p, a2 - p)) < 0;
}

// 多边形的有向面积
double PolygonArea(const Polygon& poly) {
    double area = 0;
    int n = poly.size();
    for (int i = 1; i < n - 1; i++)
        area += Cross(poly[i] - poly[0], poly[(i + 1) % n] - poly[0]);
    return area / 2;
}

struct Edge {
    int from, to; // 起点, 终点, 左边的面编号
    double ang;
};

const int maxn = 10000 + 10; // 最大边数

// 平面直线图 (PSLG) 实现
struct PSLG {
    int n, m, face_cnt;
    double x[maxn], y[maxn];
    vector<Edge> edges;
    vector<int> G[maxn];
    int vis[maxn * 2]; // 每条边是否已经访问过
    int left[maxn * 2]; // 左面的编号
    int prev[maxn * 2]; // 相同起点的上一条边 (即顺时针旋转碰到的下一条边) 编号

    vector<Polygon> faces;
    double area[maxn]; // 每个 polygon 的面积

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }
};

```

```

    faces.clear();
}

// 有向线段 from->to 的极角
double getAngle(int from, int to) {
    return atan2(y[to] - y[from], x[to] - x[from]);
}

void AddEdge(int from, int to) {
    edges.push_back((Edge) {from, to, getAngle(from, to)});
    edges.push_back((Edge) {to, from, getAngle(to, from)});
    m = edges.size();
    G[from].push_back(m - 2);
    G[to].push_back(m - 1);
}

// 找出 faces 并计算面积
void Build() {
    for (int u = 0; u < n; u++) {
        // 给从 u 出发的各条边按极角排序
        int d = G[u].size();
        for (int i = 0; i < d; i++)
            for (int j = i + 1; j < d; j++) // 这里偷个懒，假设从每个点出发的线段不会太多
                if (edges[G[u][i]].ang > edges[G[u][j]].ang) swap(G[u][i], G[u][j]);
        for (int i = 0; i < d; i++)
            prev[G[u][(i + 1) % d]] = G[u][i];
    }

    memset(vis, 0, sizeof(vis));
    face_cnt = 0;
    for (int u = 0; u < n; u++)
        for (int i = 0; i < G[u].size(); i++) {
            int e = G[u][i];
            if (!vis[e]) { // 逆时针找圈
                face_cnt++;
                Polygon poly;
                for (;;) {
                    vis[e] = 1; left[e] = face_cnt;
                    int from = edges[e].from;
                    poly.push_back(Point(x[from], y[from]));
                    e = prev[e ^ 1];
                    if (e == G[u][i]) break;
                    assert(vis[e] == 0);
                }
                faces.push_back(poly);
            }
        }

    for (int i = 0; i < faces.size(); i++)

```

```

        area[i] = PolygonArea(faces[i]);
    }
};

PSLG g;
const int maxp = 100 + 5;
int n, c;
Point P[maxp], V[maxp * (maxp - 1) / 2 + maxp];
// 在V数组里找到点p
int ID(const Point& p) { return lower_bound(V, V + c, p) - V; }

// 假定poly 没有相邻点重合的情况，只需要删除3点共线的情况
Polygon simplify(const Polygon& poly) {
    Polygon ans;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        Point a = poly[i];
        Point b = poly[(i + 1) % n];
        Point c = poly[(i + 2) % n];
        if (dcmp(Cross(a - b, c - b)) != 0) ans.push_back(b);
    }
    return ans;
}

void build_graph() {
    c = n;
    for (int i = 0; i < n; i++)
        V[i] = P[i];

    vector<double> dist[maxp]; // dist[i][j] 是第 i 条线段上第 j 个点离起点 (P[i]) 的距离
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (SegmentProperIntersection(P[i], P[(i + 1) % n], P[j], P[(j + 1) % n]))
            {
                Point p = GetLineIntersection(P[i], P[(i + 1) % n] - P[i], P[j], P[(j + 1) % n] - P[j]);
                V[c++] = p;
                dist[i].push_back(Length(p - P[i]));
                dist[j].push_back(Length(p - P[j]));
            }

    /* 为了保证“很接近的点”被看作同一个，这里使用了 sort+unique 的方法
    必须使用前面提到的“理论上是错误”的小于运算符，否则不能保证“很接近的点”在排序后能连续排列
    另一个常见的处理方式是坐标扩大很多倍（如 100000 倍），然后四舍五入变成整点（计算完后再还原），用少许的精度损失换取鲁棒性和速度
    */
    sort(V, V + c);
    c = unique(V, V + c) - V;
}

```

```

g.init(c); // c是平面图的点数
for (int i = 0; i < c; i++) {
    g.x[i] = V[i].x;
    g.y[i] = V[i].y;
}
for (int i = 0; i < n; i++) {
    Vector v = P[(i + 1) % n] - P[i];
    double len = Length(v);
    dist[i].push_back(0);
    dist[i].push_back(len);
    sort(dist[i].begin(), dist[i].end());
    int sz = dist[i].size();
    for (int j = 1; j < sz; j++) {
        Point a = P[i] + v * (dist[i][j - 1] / len);
        Point b = P[i] + v * (dist[i][j] / len);
        if (a == b) continue;
        g.AddEdge(ID(a), ID(b));
    }
}

g.Build();

Polygon poly;
for (int i = 0; i < g.faces.size(); i++)
    if (g.area[i] < 0) { // 对于连通图，只有一个面积小于零的面是无限面
        poly = g.faces[i];
        reverse(poly.begin(), poly.end()); // 对内部区域，无限面多边形的各个顶点是顺时针的
        poly = simplify(poly); // 无限面多边形上可能会有相邻共线点
        break;
    }

int m = poly.size();
printf("%d\n", m);

// 挑选坐标最小的点作为输出的起点
int start = 0;
for (int i = 0; i < m; i++)
    if (poly[i] < poly[start]) start = i;
for (int i = start; i < m; i++)
    printf("%.4lf %.4lf\n", poly[i].x, poly[i].y);
for (int i = 0; i < start; i++)
    printf("%.4lf %.4lf\n", poly[i].x, poly[i].y);
}

int main() {
    while (scanf("%d", &n) == 1 && n) {
        for (int i = 0; i < n; i++)
            scanf("%d%d", &x, &y), P[i] = Point(x, y);
        build_graph();
    }
}

```